Исполняемый файл еще не процесс

Путь от готового файла до виртуальной памяти, загрузчика и точки входа Go.

```
файл → ELF / Mach-O
execve → отображения памяти
динамическая линковка → вход в рантайм
```

Почему `./app` не означает немедленный `main.main`

УПРОЩЕНИЕ

Упрощение: ОС прочитала двоичный файл и вызвала пользовательский `main`.

ТОЧНАЯ МОДЕЛЬ

Реальность: ядро создает адресное пространство, отображает сегменты и подготавливает стартовый стек. Иногда еще работает динамический загрузчик.

ИНЖЕНЕРНЫЙ ВЫВОД

Первый исполняемый адрес программы и `main.main` — разные точки.

Вся цепочка запуска

КАРТА ЛЕКЦИИ

```
готовый исполняемый файл
↓ `execve`
ядро читает ELF / Mach-O
↓ отображает сегменты
стартовый стек и ASLR
↓ динамический загрузчик при необходимости
точка входа
↓ запуск рантайма Go
позже `main.main`
```

ИНЖЕНЕРНЫЙ ВЫВОД

Задача 4.5 — дойти от файла до старта рантайма, не повторяя его внутреннюю инициализацию.

Что мы уже знаем о виртуальной памяти

УЖЕ РАЗОБРАНО

Из блока 3.x известны страницы, таблицы страниц, ошибка страницы и резидентность.

НОВЫЙ ВОПРОС

Теперь спрашиваем не «как виртуальный адрес попадает в RAM», а «кто и по каким правилам создал эти виртуальные диапазоны процесса».

ИНЖЕНЕРНЫЙ ВЫВОД

ELF/Mach-O — источник описания будущих отображений памяти.

Исполняемый файл — не массив машинных инструкций

УПРОЩЕНИЕ

Слишком простая модель: файл состоит из инструкций, которые CPU последовательно выполняет.

ТОЧНАЯ МОДЕЛЬ

Нужны заголовок, ISA, точка входа, данные, описание сегментов, права R/W/X и сведения о библиотеках.

ИНЖЕНЕРНЫЙ ВЫВОД

Файл несет не только код, но и рецепт его размещения.

От файла к виртуальным диапазонам

Почему загрузчик читает сегменты, как возникают `.bss` и права памяти.

```
заголовок ELF → программные заголовки
PT_LOAD → адреса / смещения
размер файла ≠ размер памяти
```

ELF описывает несколько видов двоичных файлов

ФОРМАТ

На Linux ELF используется для исполняемых файлов, общих библиотек, объектных файлов и дампов памяти.

НАЗНАЧЕНИЕ

Одинаковый внешний формат не означает, что их одинаково загружают: для процесса важны записи о сегментах.

ИНЖЕНЕРНЫЙ ВЫВОД

ELF — Executable and Linkable Format.

ELF начинается с заголовка

ELF HEADER

```
магические байты `7F 45 4C 46`
↓ класс 32/64 и порядок байтов
↓ тип файла и архитектура
↓ адрес точки входа
↓ расположение таблиц
дальше — программные заголовки / секции
```

ИНЖЕНЕРНЫЙ ВЫВОД

Без заголовка ядро не может корректно интерпретировать содержимое файла.

Магическая последовательность ELF

GO / КОМАНДА

```
7F 45 4C 46
\x7F E L F
```

ЧТО ПРОИСХОДИТ

Первые четыре байта позволяют распознать формат. Если ядро не узнает ELF, могут рассматриваться другие обработчики, например сценарий с `#!`.

ИНЖЕНЕРНЫЙ ВЫВОД

Распознавание формата происходит раньше запуска кода.

e_type: исполняемый файл или разделяемый объект

ТИП ELF

`ET_EXEC`
→ традиционный исполняемый файл

`ET_DYN`
→ общий объект или PIE
→ допускает перемещаемую базу

ИНЖЕНЕРНЫЙ ВЫВОД

ET_DYN не обязательно означает «только библиотека»: PIE тоже может иметь этот тип.

`e_machine` ограничивает архитектуру CPU

ЗАДАЧА

Заголовок содержит ISA, под которую собран машинный код: например x86-64 или AArch64.

МЕХАНИЗМ

Ядро не умеет просто исполнять инструкции arm64 на процессоре amd64; для этого нужен отдельный слой трансляции.

ИНЖЕНЕРНЫЙ ВЫВОД

Формат и архитектура проверяются до передачи управления программе.

Точка входа — не `main.main`

УПРОЩЕНИЕ

`e_entry` — поле адреса, с которого начинается исполнение пользовательского кода после подготовки образа.

ТОЧНАЯ МОДЕЛЬ

В Go этот адрес ведет в стартовый код рантайма. `main.main` будет вызван существенно позднее.

ИНЖЕНЕРНЫЙ ВЫВОД

Запуск Go-программы имеет скрытый этап инициализации.

Секции и сегменты нельзя смешивать

УПРОЩЕНИЕ

Секции `.text`, `.rodata`, `.data`, `.bss` полезны компоновщику, ассемблеру и отладчикам.

ТОЧНАЯ МОДЕЛЬ

Загрузчик ядра прежде всего работает с программными заголовками, которые описывают сегменты и будущие отображения.

ИНЖЕНЕРНЫЙ ВЫВОД

Секции — удобная структура файла; сегменты — инструкция загрузчику.

Почему загрузчику удобнее сегменты

SECTIONS SEGMENTS

```
секции: `.text` `.rodata` `.data` `.bss`
↓ сгруппировать по назначению
сегмент только чтение
сегмент чтение + исполнение
сегмент чтение + запись
↓ создать отображения памяти
```

ИНЖЕНЕРНЫЙ ВЫВОД

ОС нужны крупные виртуальные диапазоны с конкретными правами, а не отдельная операция для каждой секции.

таблица программных заголовков — рецепт загрузки

ЗАДАЧА

Записи описывают тип сегмента, смещение внутри файла, виртуальный адрес, размеры и права.

МЕХАНИЗМ

Загрузчик получает достаточно сведений, чтобы создать отображения без выполнения пользовательского кода.

ИНЖЕНЕРНЫЙ ВЫВОД

Главный объект для изучения запуска ELF — таблица программных заголовков.

PT_LOAD превращается в отображение

ЗАГРУЖАЕМЫЙ СЕГМЕНТ

```
`PT_LOAD`
↓ смещение `p_offset` в файле
↓ виртуальный адрес `p_vaddr`
↓ размер `p_filesz` / `p_memsz`
↓ права `p_flags`
отображение в адресном пространстве
```

ИНЖЕНЕРНЫЙ ВЫВОД

PT_LOAD — физический мост от данных исполняемого файла к виртуальной памяти процесса.

Пять полей PT_LOAD, которые надо понимать

ЗАДАЧА

`p_offset` — смещение в файле;
`p_vaddr` — виртуальный адрес;
`p_filesz` — число байт в файле.

МЕХАНИЗМ

`p_memsz` — размер в памяти;
`p_flags` — права чтения, записи и исполнения.

ИНЖЕНЕРНЫЙ ВЫВОД

Из этой пятерки объясняется почти вся базовая картина загрузки ELF.

Сопоставляем смещение в файле и виртуальный адрес

ФАЙЛ ПАМЯТЬ

```
ELF: байты `[offset X .. Y)`
↓ `PT_LOAD`
виртуальный диапазон `[A .. B)`
↓ страницы пока могут быть не в RAM
обращение CPU
↓ знакомый механизм ошибка страницы
```

ИНЖЕНЕРНЫЙ ВЫВОД

Само отображение не означает немедленную загрузку всех физических страниц.

Почему `p_memsz` может превышать `p_filesz`

УПРОЩЕНИЕ

В файле есть, допустим, 100 КиБ данных, но сегмент должен занимать 10 МиБ памяти.

ТОЧНАЯ МОДЕЛЬ

Разница имеет нулевое начальное содержимое. Держать миллионы нулевых байт на диске нет необходимости.

ИНЖЕНЕРНЫЙ ВЫВОД

Различие размеров особенно важно для `.bss`.

`.bss`: большой нулевой массив

GO / КОМАНДА

```
var buf [1 << 30]byte
```

ЧТО ПРОИСХОДИТ

Логический глобальный объект занимает 1 ГиБ, но файл не обязан включать 1 ГиБ нулей. Виртуальная память может быть создана и заполнена нулями по требованию.

ИНЖЕНЕРНЫЙ ВЫВОД

Нулевая инициализация и хранение байтов в файле — разные задачи.

`mapped` не означает `resident`

УПРОЩЕНИЕ

После загрузки большой сегмент уже имеет виртуальный диапазон и права доступа.

ТОЧНАЯ МОДЕЛЬ

Физические страницы могут появляться только при первом обращении. Резидентность зависит от страничный кеш и механизма виртуальной памяти.

ИНЖЕНЕРНЫЙ ВЫВОД

Не выводим RSS из `p_memsz` или размера бинарника.

Почему код обычно R-X, а данные — RW-

ПРАВА СТРАНИЦ

```
флаги `PT_LOAD`
├─ код → `R-X`
├─ константы → `R--`
└─ изменяемые данные → `RW-`
↓ таблицы страниц
↓ контроль доступа через MMU
```

ИНЖЕНЕРНЫЙ ВЫВОД

Права выполнения рождаются из метаданных файла и реально применяются аппаратурой.

Вся память процесса — не «просто байты»

УПРОЩЕНИЕ

Ошибочная модель: если известен адрес, код может свободно читать, менять и выполнять любые байты.

ТОЧНАЯ МОДЕЛЬ

Страницы имеют независимые права. `W^X` стремится исключить ситуацию, когда одна страница одновременно записываемая и исполняемая.

ИНЖЕНЕРНЫЙ ВЫВОД

Настройки загрузчика участвуют в защите процесса.

Зачем нужна защита W^X

ЗАДАЧА

Если атакующий может изменить исполняемую страницу, внедрение собственного машинного кода существенно упрощается.

МЕХАНИЗМ

Разделение записываемых данных и исполняемого кода уменьшает поверхность подобных атак.

ИНЖЕНЕРНЫЙ ВЫВОД

W^X — защитная мера, а не исправление ошибок памяти.

ELF описывает будущее адресное пространство

МОДЕЛЬ ЗАГРУЗКИ

```
заголовок ELF
↓ программные заголовки
сегменты / размеры / права
↓ загрузчик ядра
виртуальные отображения
↓ MMU / загрузка страниц по требованию
физическая резидентность по мере доступа
```

ИНЖЕНЕРНЫЙ ВЫВОД

Формат файла задает структуру памяти, а не просто список инструкций.

`readelf -l`: взгляд загрузчика

GO / КОМАНДА

```
readelf -l ./app
```

ЧТО ПРОИСХОДИТ

Показывает `LOAD`, `INTERP`, `DYNAMIC`, `GNU_STACK` и `GNU_RELRO`. Это главная команда для выяснения будущих отображений процесса.

ИНЖЕНЕРНЫЙ ВЫВОД

Нас интересуют программные заголовки, а не только перечень секций.

`readelf -S`: взгляд компоновщика

GO / КОМАНДА

```
readelf -S ./app
```

ЧТО ПРОИСХОДИТ

Показывает `.text`, `.rodata`, `.data`, `.bss` и остальные секции. Сопоставьте их с `PT_LOAD` из предыдущей команды.

ИНЖЕНЕРНЫЙ ВЫВОД

Разница между секциями и сегментами должна быть видна руками.

`readelf -h`: тип, архитектура и точка входа

GO / КОМАНДА

```
readelf -h ./app
```

ЧТО ПРОИСХОДИТ

Позволяет проверить `ET_EXEC`/`ET_DYN`, `e_machine`, класс ELF и `e_entry`.

ИНЖЕНЕРНЫЙ ВЫВОД

Три команды `-h`, `-l`, `-S` дают базовую карту исполняемого файла.

Как программа получает новый образ процесса

Процесс не создается самим `execve`; его адресное пространство заменяется.

```
PID сохраняется
argv / envp / auxv
страницы исполняемого файла
```

Что скрывается за запуском `./app`

ЗАПУСК

```
оболочка запускает программу
↓ создание дочернего процесса, если нужно
системный вызов `execve`
↓ замена исполняемого образа
новые отображения и начальный стек
↓ переход в точку входа
```

ИНЖЕНЕРНЫЙ ВЫВОД

Создание процесса и замена исполняемого образа — разные системные операции.

`execve` не создает новый PID

УПРОЩЕНИЕ

Плохая модель: `execve` создал новый процесс, потому что после него выполняется другая программа.

ТОЧНАЯ МОДЕЛЬ

Процесс уже существует. `execve` заменяет его пользовательский образ; PID при успешном вызове сохраняется.

ИНЖЕНЕРНЫЙ ВЫВОД

> Не смешиваем `fork`/`clone` с `execve`.

Что меняет execve

НОВЫЙ ОБРАЗ

```
старые пользовательские отображения
↓ заменить образ
код и данные нового ELF
↓ новый стек запуска
`argc` / `argv` / `envp` / `auxv`
↓ новый указатель команд
```

ИНЖЕНЕРНЫЙ ВЫВОД

Часть атрибутов процесса сохраняется, другие сбрасываются по правилам ОС.

Почему дескрипторы могут пережить запуск

ЗАДАЧА

По умолчанию открытый файловый дескриптор способен сохраниться при `execve`.

МЕХАНИЗМ

Флаг `FD_CLOEXEC` требует закрытия дескриптора при успешной замене образа.

ИНЖЕНЕРНЫЙ ВЫВОД

Наследование дескрипторов — часть контракта безопасности процесса.

Без CLOEXEC дочерняя программа может получить лишний доступ

УПРОЩЕНИЕ

Сервер открыл файловый дескриптор с чувствительными данными и запускает вспомогательную программу.

ТОЧНАЯ МОДЕЛЬ

Если дескриптор наследуется, новая программа получает возможность обращаться к ресурсу. FD_CLOEXEC разрывает эту связь.

ИНЖЕНЕРНЫЙ ВЫВОД

Права доступа передаются также через открытые файловые дескрипторы.

Три аргумента `execve`

ЗАДАЧА

Путь исполняемого файла, массив `argv` и массив переменных окружения `envp`.

МЕХАНИЗМ

Рантайм Go читает эти данные уже после передачи управления загрузчиком, чтобы позже сформировать `os.Args`.

ИНЖЕНЕРНЫЙ ВЫВОД

Параметры Go-программы имеют происхождение в ABI операционной системы.

Что находится на стартовом стеке Linux

СТЕК ЗАПУСКА

```
начальный стек ОС
├─ `argc`
├─ указатели `argv[]`
├─ указатели `envp[]`
├─ пары `auxv[]`
└─ строки аргументов и окружения
```

ИНЖЕНЕРНЫЙ ВЫВОД

Стек создается ядром до первой инструкции рантайма Go.

Как возникает `os.Args`

АРГУМЕНТЫ

`execve(path, argv, envp)`
↓ ядро формирует начальный стек
стартовый код рантайма
↓ разбирает `argc/argv`
пакет `os`
↓ `os.Args`

ИНЖЕНЕРНЫЙ ВЫВОД

API Go использует данные, физически переданные через системный ABI.

auxv: дополнительные сведения от ядра

ЗАДАЧА

Вектор auxv состоит из пар «тип — значение». Он передает данные, полезные стартовому коду.

МЕХАНИЗМ

Примеры: размер страницы, расположение программных заголовков, случайные байты, данные платформы и адрес vDSO.

ИНЖЕНЕРНЫЙ ВЫВОД

Не все стартовые параметры нужно выяснять дополнительными системными вызовами.

Зачем auxv экономит работу

KERNEL USER SPACE

```
ядро уже знает сведения о запуске
↓ кладет пары `auxv` на стек
динамический загрузчик / рантайм
↓ читает готовые значения
меньше запросов к ядру в начале
```

ИНЖЕНЕРНЫЙ ВЫВОД

auxv — канал передачи метаданных при входе в пользовательский код.

AT_RANDOM появляется еще до main

ЗАДАЧА

auxv может передавать случайные байты, предоставленные ядром.

МЕХАНИЗМ

Они используются отдельными защитными инициализациями пользовательского окружения, когда это предусматривает платформа.

ИНЖЕНЕРНЫЙ ВЫВОД

Часть параметров безопасности готова до Go начало работы рантайма.

vDSO — не часть Go-бинарника

УПРОЩЕНИЕ

Кажется, что каждый пользовательский вызов системной функции обязан выполнять полноценный переход в ядро.

ТОЧНАЯ МОДЕЛЬ

Linux дополнительно отображает vDSO — небольшой код ядра для некоторых операций на стороне пользователя.

ИНЖЕНЕРНЫЙ ВЫВОД

Не всякая услуга ядра требует отдельного системного вызова.

Зачем нужен vDSO

ОПТИМИЗАЦИЯ ПЕРЕХОДА

```
обычный путь:
пользователь → syscall → ядро → возврат

для части операций:
пользователь → вызов vDSO
↓ используется отображенный код / данные
```

ИНЖЕНЕРНЫЙ ВЫВОД

vDSO уменьшает стоимость некоторых часто используемых системных операций.

Находим vDSO в памяти процесса

GO / КОМАНДА

```
cat /proc/$PID/maps
```

ЧТО ПРОИСХОДИТ

В списке отображений можно увидеть `[vdso]`, `[vvar]` и другие специальные диапазоны. Это отображения, добавленные ядром, а не секции вашего ELF.

ИНЖЕНЕРНЫЙ ВЫВОД

Карта памяти процесса шире его исполняемого файла.

Обычные отображения предыдущей программы исчезают

ЗАДАЧА

После успешного `execve` старый пользовательский код и данные заменяются отображениями нового образа.

МЕХАНИЗМ

Новый адресный план строится по формату исполняемого файла; старое содержимое пользовательского пространства не переносится как есть.

ИНЖЕНЕРНЫЙ ВЫВОД

`execve` — реальная замена программы внутри уже существующего процесса.

При execve ядро не читает сразу весь код в RAM

УПРОЩЕНИЕ

Наивная схема: прочитать весь ELF, скопировать все байты в оперативную память и только затем запустить.

ТОЧНАЯ МОДЕЛЬ

На практике создаются файловые отображения. Страницы кода могут быть загружены по требованию.

ИНЖЕНЕРНЫЙ ВЫВОД

Размер ELF и объем стартового чтения с диска не равны.

Первое исполнение страницы кода

ЛЕНИВАЯ ЗАГРУЗКА

```
сегмент отображен виртуально
↓ нужной страницы пока нет в RAM
CPU выбирает инструкцию
↓ ошибка страницы
страничный кеш / накопитель
↓ страница становится резидентной
инструкция повторяется
```

ИНЖЕНЕРНЫЙ ВЫВОД

Знакомый загрузка страниц по требованию применяется и к исполняемым страницам.

Почему 100 МиБ ELF не означают 100 МиБ чтения при старте

УПРОЩЕНИЕ

Ошибочная модель: перед запуском нужно считать с диска весь исполняемый файл.

ТОЧНАЯ МОДЕЛЬ

Программа может коснуться лишь части страниц кода и метаданных. Остальные останутся только отображенными.

ИНЖЕНЕРНЫЙ ВЫВОД

Стартовая нагрузка определяется реально востребованными страницами.

Холодный и прогретый запуск

ЗАДЕРЖКА ЗАПУСКА

```
первый запуск
↓ холодные страницы кода и библиотек
↓ ошибки страниц и возможный ввод с диска

повторный запуск
↓ данные уже в страничном кеше
↓ дешевле удовлетворить обращения
```

ИНЖЕНЕРНЫЙ ВЫВОД

Холодный запуск и работа с прогретым кешем измеряются отдельно.

Page faults на старте не обязательно проблема

ЗАДАЧА

Незначительные ошибки страниц ожидаемы при работе с файловыми отображениями и нулевыми страницами.

МЕХАНИЗМ

Диагностируем их как источник задержки только после измерений, когда ввод-вывод или обработка ошибок страниц действительно заметны.

ИНЖЕНЕРНЫЙ ВЫВОД

Сам счетчик ошибки страниц не доказывает неисправность.

Контейнер не получает собственное ядро и загрузчик

УПРОЩЕНИЕ

Ошибка: раз бинарник внутри контейнера, его запускает особая «контейнерная ОС».

ТОЧНАЯ МОДЕЛЬ

Linux-контейнер использует ядро хоста, включая обработчик ELF. Меняются пространства имен, cgroup, видимая файловая система и полномочия.

ИНЖЕНЕРНЫЙ ВЫВОД

Физический механизм `execve` остается системным вызовом ядра Linux.

Почему библиотека внутри контейнера не совпадает с библиотекой хоста

ЗАДАЧА

Динамический загрузчик видит пути внутри файлового пространства имен процесса.

МЕХАНИЗМ

Проверка библиотек на хосте может дать другой результат, чем проверка в контейнере с отдельной корневой файловой системой.

ИНЖЕНЕРНЫЙ ВЫВОД

Зависимости загрузчика диагностируем в том же пространстве имен, где запускается сервис.

Почему адрес функции меняется между запусками

ASLR рандомизирует размещение, PIE позволяет перемещать основной исполняемый образ.

```
ET_EXEC / ET_DYN
база загрузки + относительное смещение
позиционно-независимый код
```

Фиксированные адреса упрощают эксплуатацию ошибок

УПРОЩЕНИЕ

Слишком простая модель: функция всегда располагается по одному адресу в виртуальной памяти.

ТОЧНАЯ МОДЕЛЬ

ASLR изменяет размещение ключевых областей процесса, повышая непредсказуемость абсолютных адресов.

ИНЖЕНЕРНЫЙ ВЫВОД

ASLR затрудняет некоторые атаки, но не устраняет ошибки памяти.

Что может перемещать ASLR

АДРЕСНОЕ ПРОСТРАНСТВО

```
запуск №1 → база A
запуск №2 → база B

области с меняющейся базой:
исполняемый образ / библиотеки
стек / хип / mmap-диапазоны
```

ИНЖЕНЕРНЫЙ ВЫВОД

Политика рандомизации зависит от ОС, типа файла и настроек безопасности.

Почему классический ET_EXEC менее гибок

ЗАДАЧА

Традиционный исполняемый файл часто связан под заранее определенные виртуальные адреса.

МЕХАНИЗМ

Чтобы перемещать весь образ, машинный код и адресные ссылки должны допускать изменение базового адреса.

ИНЖЕНЕРНЫЙ ВЫВОД

Формат и стратегия кодогенерации определяют свободу ASLR.

PIE и ASLR — не одно и то же

УПРОЩЕНИЕ

ASLR — политика рандомизации адресов при запуске.

ТОЧНАЯ МОДЕЛЬ

PIE — позиционно-независимая форма исполняемого файла, позволяющая загрузить основной образ по разным базовым адресам.

ИНЖЕНЕРНЫЙ ВЫВОД

PIE помогает применять ASLR к основному исполняемому файлу.

Адрес функции в PIE

ФОРМУЛА АДРЕСА

```
функция F имеет относительное смещение D
↓ загрузчик выбирает базу B
реальный адрес F = B + D
↓ следующий запуск: база C
реальный адрес F = C + D
```

ИНЖЕНЕРНЫЙ ВЫВОД

Меняется база загрузки, а внутренние смещения функций сохраняют смысл.

Что делает позиционно-независимый код

ЗАДАЧА

Там, где возможно, используются относительная адресация, смещения от счетчика команд и таблицы косвенных ссылок.

МЕХАНИЗМ

Это снижает число мест, которым нужен жестко вшитый абсолютный адрес.

ИНЖЕНЕРНЫЙ ВЫВОД

Позиционно-независимый код облегчает перемещение образа.

PIE не обязан заметно замедлить сервер

УПРОЩЕНИЕ

Можно ожидать, что дополнительные относительные вычисления и косвенные обращения обязательно ухудшат производительность.

ТОЧНАЯ МОДЕЛЬ

Современные ISA и компиляторы хорошо поддерживают такую адресацию. Реальная цена зависит от кода и сборки.

ИНЖЕНЕРНЫЙ ВЫВОД

Нужны измерения, а не оценка только по типу ELF.

Проверяем PIE и ASLR

GO / КОМАНДА

```
readelf -h ./app
cat /proc/$PID/maps
```

ЧТО ПРОИСХОДИТ

Тип `ET_DYN` характерен для PIE. Несколько запусков позволяют сравнить виртуальные базовые адреса основного образа.

ИНЖЕНЕРНЫЙ ВЫВОД

ASLR удобно показывать на одном и том же бинарнике в разных процессах.

Кто подставляет адреса внешних функций

Статическая сборка решает ссылки заранее, динамическая часть работы остается загрузчику.

```
PT_INTERP → ld-linux
DT_NEEDED → .so
relocation → GOT/PLT
```

Статическая программа может идти сразу к точке входа

УПРОЩЕНИЕ

В простейшем статическом ELF весь необходимый машинный код уже находится в исполняемом файле.

ТОЧНАЯ МОДЕЛЬ

Ядро создает отображения, готовит стек и передает управление, не привлекая пользовательский загрузчик общих библиотек.

ИНЖЕНЕРНЫЙ ВЫВОД

Статическая линковка сокращает число внешних зависимостей запуска.

Зачем понадобились общие библиотеки

ЗАДАЧА

Несколько процессов могут использовать код из одного `.so`; чистые страницы с кодом способны разделяться физически.

МЕХАНИЗМ

Библиотеку можно обновлять независимо от приложения при совместимости ABI.

ИНЖЕНЕРНЫЙ ВЫВОД

Переиспользование кода имеет цену загрузки, разрешения символов и управления зависимостями.

Общие библиотеки одновременно помогают и усложняют запуск

УПРОЩЕНИЕ

Плюсы: меньше повторов кода и централизованное обновление совместимой библиотеки.

ТОЧНАЯ МОДЕЛЬ

Цена: дополнительная работа при запуске, поиск нужных версий и риск несовместимости в окружении.

ИНЖЕНЕРНЫЙ ВЫВОД

Выбор статической или динамической линковки — эксплуатационный компромисс.

PT_INTERP: путь к динамическому загрузчику

ПРОМЕЖУТОЧНЫЙ ЗАГРУЗЧИК

```
ELF программные заголовки
↓ запись `PT_INTERP`
путь `/lib64/ld-linux-...`
↓ ядро отображает интерпретатор
управление сначала получает `ld-linux`
↓ затем стартовая точка программы
```

ИНЖЕНЕРНЫЙ ВЫВОД

Динамический загрузчик запускается ядром как пользовательский код.

Динамический загрузчик не является частью ядра

УПРОЩЕНИЕ

Плохая модель: ядро само находит все `.so`, разрешает символы и непосредственно вызывает `main`.

ТОЧНАЯ МОДЕЛЬ

Ядро загружает интерпретатор ELF; дальше `ld-linux` в пользовательском пространстве настраивает библиотеки и их адреса.

ИНЖЕНЕРНЫЙ ВЫВОД

Различаем загрузчик ядра и пользовательский динамический загрузчик.

Что делает `ld-linux` до программы

РАБОТА ЗАГРУЗЧИКА

```
прочитать динамические метаданные
↓ найти `DT_NEEDED` библиотеки
↓ отобразить их сегменты
↓ разрешить символы
↓ применить релокации и защиты
передать управление точке входа
```

ИНЖЕНЕРНЫЙ ВЫВОД

Динамическая линковка — отдельный исполняемый этап до запуска приложения.

PT_DYNAMIC: таблица оставшейся работы

ЗАДАЧА

Динамические записи могут включать DT_NEEDED, DT_STRTAB, DT_SYMTAB, DT_RELA/DT_REL и DT_RUNPATH.

МЕХАНИЗМ

Они сообщают загрузчику, какие библиотеки, имена, символы и релокации нужны.

ИНЖЕНЕРНЫЙ ВЫВОД

Динамический раздел является входными данными пользовательского загрузчика.

DT_NEEDED строит граф зависимостей

ЗАВИСИМОСТИ

```
исполняемый файл
├─ `DT_NEEDED`: библиотека A
├─ `DT_NEEDED`: библиотека B
↓ A / B тоже могут требовать библиотеки
граф загрузки `.so`
```

ИНЖЕНЕРНЫЙ ВЫВОД

Нужно удовлетворить транзитивные зависимости, а не только найти одну библиотеку.

`readelf -d` показывает динамические зависимости

GO / КОМАНДА

```
readelf -d ./app
```

ЧТО ПРОИСХОДИТ

В первую очередь смотрим `NEEDED`, `RUNPATH`, `RPATH` и флаги. Отсутствие `DT_NEEDED` обычно означает отсутствие привычных динамических зависимостей.

ИНЖЕНЕРНЫЙ ВЫВОД

Не гадаем о линковке по языку исходников — проверяем сам ELF.

`ldd`: только для доверенных программ

GO / КОМАНДА

```
ldd ./app
readelf -d ./app
```

ЧТО ПРОИСХОДИТ

`ldd` показывает разрешенные зависимости; для недоверенного бинарника предпочитаем статический просмотр через `readelf`, поскольку поведение `ldd` зависит от реализации.

ИНЖЕНЕРНЫЙ ВЫВОД

Диагностический инструмент не должен бесконтрольно выполнять чужой код.

Как загрузчик находит `.so`

ПОИСК БИБЛИОТЕКИ

```
имя в `DT_NEEDED`
↓ поиск по применимым путям
`RPATH` / `RUNPATH` / окружение
↓ системный кеш и каталоги
найденный файл библиотеки
↓ отобразить ее `PT_LOAD`
```

ИНЖЕНЕРНЫЙ ВЫВОД

Точные приоритеты поиска зависят от платформы и динамического загрузчика.

RUNPATH — реальная проблема поставки

ЗАДАЧА

Путь к библиотеке на машине разработчика не обязан существовать в образе контейнера.

МЕХАНИЗМ

Встроенный RUNPATH, системные каталоги и переменные окружения участвуют в поиске. Нужный файл должен быть доступен в правильном пространстве имен.

ИНЖЕНЕРНЫЙ ВЫВОД

Ошибка загрузки `.so` может не иметь отношения к Go-коду.

`.so` тоже содержит ELF-заголовки и сегменты

ЗАДАЧА

Общая библиотека имеет свои `PT_LOAD`, таблицу символов, динамический раздел и релокации.

МЕХАНИЗМ

Загрузчик отображает ее страницы в адресном пространстве процесса и назначает фактическую базу с учетом ASLR.

ИНЖЕНЕРНЫЙ ВЫВОД

Библиотека — отдельный загружаемый образ с собственным набором ссылок.

Адрес библиотеки определяется во время запуска

БАЗА БИБЛИОТЕКИ

```
`.so` описывает относительную раскладку
↓ ASLR назначает базу B
фактические адреса = B + смещения
↓ остаются динамические ссылки
динамические релокации
```

ИНЖЕНЕРНЫЙ ВЫВОД

Компоновщик не всегда знает окончательные адреса внешних символов.

Не все релокации разрешаются в 4.4

УПРОЩЕНИЕ

Статический компоновщик связывает известные ему символы и размещает код в готовом файле.

ТОЧНАЯ МОДЕЛЬ

Адрес функции в `.so` может быть неизвестен до запуска. Его подставляет уже динамический загрузчик.

ИНЖЕНЕРНЫЙ ВЫВОД

Статическая и динамическая релокация происходят на разных стадиях.

Как работает динамическая релокация

РЕЛОКАЦИЯ

```
в ELF есть ссылка на символ F
↓ библиотека загружена по базе B
динамический загрузчик находит F
↓ вычисляет окончательный адрес
записывает адрес в нужную ячейку
```

ИНЖЕНЕРНЫЙ ВЫВОД

Символьная ссылка превращается в фактический адрес во время запуска.

Почему не стоит менять страницы машинного кода

УПРОЩЕНИЕ

Наивно: переписывать адрес каждой внешней функции прямо в инструкциях `.text`.

ТОЧНАЯ МОДЕЛЬ

Это требует записи в исполняемые страницы и мешает защите и физическому разделению чистого кода.

ИНЖЕНЕРНЫЙ ВЫВОД

Часть изменяемых адресов выгодно вынести в отдельные таблицы.

GOT хранит адреса, известные только после загрузки

GOT

```
код обращается к ячейке GOT
↓ в ней адрес символа
динамический загрузчик может заменить адрес
↓ инструкция в `.text` не меняется
```

ИНЖЕНЕРНЫЙ ВЫВОД

таблица глобальных смещений (Global Offset Table) отделяет исполняемые инструкции от адресов, которые нужно заполнить позднее.

PLT организует вызов внешней функции

PLT

```
`CALL F@PLT`
↓ локальный участок PLT
↓ адрес из GOT
↓ фактическая функция F
```

ИНЖЕНЕРНЫЙ ВЫВОД

таблица переходов к процедурам (Procedure Linkage Table) позволяет переходить к динамически найденному коду.

GOT/PLT — не поиск по имени при каждом вызове

УПРОЩЕНИЕ

Плохая модель: при каждом обращении к внешней функции загрузчик снова ищет строковое имя.

ТОЧНАЯ МОДЕЛЬ

После связывания GOT содержит готовый адрес, а PLT направляет вызов по нему.

ИНЖЕНЕРНЫЙ ВЫВОД

Динамический вызов обычно не повторяет полный поиск символа.

Ленивое связывание: первый вызов дороже

LAZY BINDING

```
первый `CALL F@PLT`
↓ GOT еще ведет в resolver
загрузчик ищет символ F
↓ записывает адрес в GOT
вызвать F
↓ последующие вызовы идут прямо по адресу
```

ИНЖЕНЕРНЫЙ ВЫВОД

Часть цены запуска переносится на первое использование функции.

Ленивое и раннее связывание меняют место задержки

УПРОЩЕНИЕ

Ленивое связывание: меньше работы при старте, но первый вызов внешней функции проходит через resolver.

ТОЧНАЯ МОДЕЛЬ

Раннее связывание: адреса готовятся до входа в приложение, ошибки символов проявляются раньше.

ИНЖЕНЕРНЫЙ ВЫВОД

Задержка не исчезает — она распределяется между запуском и первым вызовом.

LD_BIND_NOW выбирает раннее связывание

GO / КОМАНДА

```
LD_BIND_NOW=1 ./app
```

ЧТО ПРОИСХОДИТ

Для glibc это заставляет заранее разрешать подходящие динамические символы. На статическом бинарнике без динамического загрузчика эффекта не будет.

ИНЖЕНЕРНЫЙ ВЫВОД

Эксперимент оценивает перенос работы из первого вызова в запуск.

LD_DEBUG показывает деятельность загрузчика

GO / КОМАНДА

```
LD_DEBUG=libs,reloc,bindings ./app
```

ЧТО ПРОИСХОДИТ

Можно увидеть поиск библиотек, релокации и связывание символов. Вывод очень подробный: использовать прежде всего в лаборатории.

ИНЖЕНЕРНЫЙ ВЫВОД

Работу загрузчика можно наблюдать, а не предполагать.

Зачем нужен RELRO

УПРОЩЕНИЕ

После применения релокаций некоторые таблицы больше не нужно менять во время работы.

ТОЧНАЯ МОДЕЛЬ

Загрузчик может защитить их от записи. Это уменьшает риск подмены адресов при повреждении памяти.

ИНЖЕНЕРНЫЙ ВЫВОД

RELRO — ограничение записи после начальной настройки адресов.

Почему полный RELRO связан с ранним связыванием

RELRO

```
ленивая связь требует записи в GOT позже
↓ полная защита не даст этого сделать
разрешить символы заранее
↓ применить релокации
`mprotect`: таблицы только для чтения
```

ИНЖЕНЕРНЫЙ ВЫВОД

Защита памяти и момент разрешения символов тесно связаны.

Как динамический загрузчик выбирает одноименные символы

ЗАДАЧА

При нескольких определениях важны область поиска, видимость, очередность загрузки и правила слабых/сильных символов.

МЕХАНИЗМ

Это позволяет подменять некоторые реализации и одновременно усложняет воспроизводимость.

ИНЖЕНЕРНЫЙ ВЫВОД

Выбор конкретного определения — отдельный механизм динамической линковки.

Подмена символов — мощный, но рискованный механизм

ЗАДАЧА

Перехват функций полезен для диагностики, совместимости и инструментирования.

МЕХАНИЗМ

Однако такая подмена может неожиданно изменить исполняемый код и создать дополнительные предположения о безопасности.

ИНЖЕНЕРНЫЙ ВЫВОД

Динамическая линковка влияет не только на запуск, но и на семантику внешних вызовов.

LD_PRELOAD добавляет определения символов

GO / КОМАНДА

```
LD_PRELOAD=./hook.so ./app
```

ЧТО ПРОИСХОДИТ

При наличии динамического загрузчика предварительно загруженная библиотека может участвовать в поиске символов и перехватывать вызовы.

ИНЖЕНЕРНЫЙ ВЫВОД

Подмена функций реализуется на стороне динамического загрузчика.

Статический бинарник не обрабатывает LD_PRELOAD обычным способом

УПРОЩЕНИЕ

Ожидание: любой процесс Linux увидит LD_PRELOAD и загрузит указанную библиотеку.

ТОЧНАЯ МОДЕЛЬ

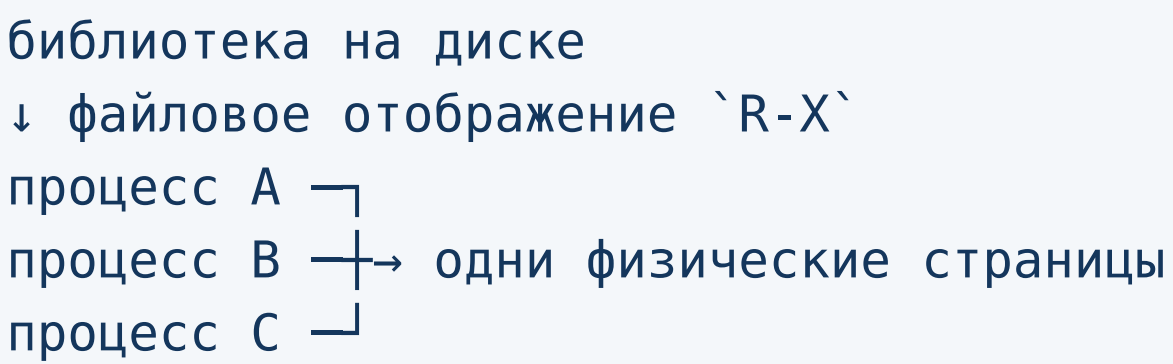
Если динамический загрузчик не вызывается, обрабатывать эту переменную просто некому.

ИНЖЕНЕРНЫЙ ВЫВОД

Переменные LD_* не универсальная магия ядра Linux.

Почему чистые страницы `.so` могут делиться

РАЗДЕЛЕНИЕ СТРАНИЦ



ИНЖЕНЕРНЫЙ ВЫВОД

Разные процессы могут разделять чистые страницы одного и того же файла.

Изменяемые данные библиотеки — другая история

УПРОЩЕНИЕ

Легко представить, будто вся память общей библиотеки обязательно одна на всех процессов.

ТОЧНАЯ МОДЕЛЬ

Глобальные изменяемые данные и страницы, измененные релокациями, требуют собственной семантики, часто частной или COW.

ИНЖЕНЕРНЫЙ ВЫВОД

Общий код не означает общих изменяемых данных.

Динамическая линковка не всегда автоматически экономит RAM

УПРОЩЕНИЕ

Сравнение только размеров `.so` и статического файла кажется достаточным.

ТОЧНАЯ МОДЕЛЬ

Нужны PSS, резидентность, частные страницы, число процессов и разделение чистого кода. Экземпляры одного статического ELF тоже делят одинаковые файловые страницы.

ИНЖЕНЕРНЫЙ ВЫВОД

Реальную память измеряем на системе, а не выводим из формата линковки.

Почему не каждый Go-бинарник статический

Один язык, но разные режимы сборки и разные зависимости во время запуска.

```
Go без cgo → часто без `PT_INTERP`
cgo → системный ABI и библиотеки
PIE → перемещаемый образ
```

«Go всегда собирает статический бинарник» — неверно

УПРОЩЕНИЕ

Обычная программа без cgo на Linux часто собирается как самодостаточный ELF через внутренний компоновщик.

ТОЧНАЯ МОДЕЛЬ

Варианты с cgo, внешней линковкой и особыми режимами сборки могут получить `PT_INTERP` и общие библиотеки.

ИНЖЕНЕРНЫЙ ВЫВОД

Свойства линковки определяются конкретным результатом сборки.

Проверяем реальную линковку Go-программы

GO / КОМАНДА

```
file ./app
readelf -l ./app
readelf -d ./app
```

ЧТО ПРОИСХОДИТ

Команды показывают формат, динамический интерпретатор и список необходимых библиотек, если они есть.

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя угадывать зависимости только по исходному языку.

Отключаем cgo для чистого Go-пути

GO / КОМАНДА

```
CGO_ENABLED=0 go build -o app .
```

ЧТО ПРОИСХОДИТ

Этот режим часто позволяет собрать Linux-программу без C-зависимостей, если все используемые пакеты имеют реализации без cgo.

ИНЖЕНЕРНЫЙ ВЫВОД

Отключение cgo меняет доступные реализации и свойства итогового файла.

cgo связывает два ABI-мира

ГРАНИЦА ABI

```
Go-код и внутренний ABI Go
↓ cgo-переход
системный C ABI
↓ объектные файлы и библиотеки C
возможна внешняя линковка
↓ динамический загрузчик при запуске
```

ИНЖЕНЕРНЫЙ ВЫВОД

cgo расширяет путь компиляции, линковки и загрузки приложения.

Почему программа запускается локально, но не в минимальном образе

УПРОЩЕНИЕ

На машине сборки есть libc и ее динамический загрузчик. Программа работает.

ТОЧНАЯ МОДЕЛЬ

В целевом контейнере интерпретатор или нужная `.so` отсутствует. Код Go не менялся, но запуск невозможен.

ИНЖЕНЕРНЫЙ ВЫВОД

Проблема может находиться ниже рантайма и приложения.

Сравниваем сборки без cgo и с cgo

GO / КОМАНДА

```
CGO_ENABLED=0 go build -o pure .
ldd ./pure

# для cgo: собрать с реальной
C-зависимостью
ldd ./dynamic
```

ЧТО ПРОИСХОДИТ

Одинаковая логика приложения может дать разные зависимости от пользовательского загрузчика и системных библиотек.

ИНЖЕНЕРНЫЙ ВЫВОД

Разницу нужно подтверждать `PT_INTERP` и `DT_NEEDED`, а не предполагать.

Go умеет собирать PIE

GO / КОМАНДА

```
go build -buildmode=pie -o
app-pie .
```

ЧТО ПРОИСХОДИТ

Позиционно-независимый исполняемый файл допускает изменение базы загрузки. Конкретные значения по умолчанию зависят от целевой платформы и режима сборки.

ИНЖЕНЕРНЫЙ ВЫВОД

Режим сборки влияет на возможную адресную раскладку процесса.

Рантайм Go тоже должен учитывать базу загрузки

ЗАДАЧА

При изменении адресов должны оставаться корректными таблицы функций, описание типов, `pclntab` и ссылки на код.

МЕХАНИЗМ

Компилятор и компоновщик согласуют адресацию и метаданные с механизмом загрузки.

ИНЖЕНЕРНЫЙ ВЫВОД

Обычный Go-код не обязан вручную пересчитывать адреса после ASLR.

Кого действительно вызывает загрузчик

После ELF или Mach-O управление получает начальный код, а не `main.main`.

```
точка входа
↓ стартовый код `rt0_*`
↓ первичная инициализация рантайма Go
```

Статический и динамический путь сходятся

ДВА ПУТИ

```
статический ELF → ядро → точка входа

динамический ELF → ядро
↓ ld-linux → релокации
↓ точка входа

далее → начальный код Go
```

ИНЖЕНЕРНЫЙ ВЫВОД

Динамическая линковка может добавиться перед точкой входа, но не заменяет инициализацию Go.

ОС не может просто вызвать `main.main`

УПРОЩЕНИЕ

`main.main` — пользовательская функция, которую хочется считать началом программы.

ТОЧНАЯ МОДЕЛЬ

До нее рантайм обязан подготовить планировщик, память, GC, G/M/P и инициализацию пакетов.

ИНЖЕНЕРНЫЙ ВЫВОД

Точка входа ELF и пользовательский `main.main` разнесены по времени.

Где появляется Go startup

ПЕРЕХОД К GO

```
ELF `e_entry`
↓ стартовый ассемблер под платформу
`rt0_*`
↓ первичная инициализация рантайма
`runtime.main`
↓ инициализация пакетов
`main.main`
```

ИНЖЕНЕРНЫЙ ВЫВОД

Названия внутренних функций могут меняться между версиями Go.

Почему внутренности `rt0` — отдельная лекция

УПРОЩЕНИЕ

4.5 объясняет, как ОС создает образ процесса и передает управление коду.

ТОЧНАЯ МОДЕЛЬ

4.6 объясняет, как начальный поток ОС превращается в `m0/g0` и как возникает рабочий планировщик Go.

ИНЖЕНЕРНЫЙ ВЫВОД

Граница проходит по передаче управления стартовому коду рантайма.

Цена процесса начинается до первого HTTP-запроса

Код, библиотеки, страницы и релокации способны увеличить задержку еще до `main.main`.

```
файловые страницы → ошибки страниц
библиотеки → новые отображения
релокации → работа CPU и памяти
```

Что уже происходит до готовности сервиса

ВРЕМЕННАЯ ШКАЛА

```
`execve` / разбор ELF
↓ отображения памяти и библиотеки
↓ релокации и защиты
↓ ошибки страниц кода
↓ первичная инициализация рантайма Go
↓ `init()` пакетов
приложение начинает слушать порт
```

ИНЖЕНЕРНЫЙ ВЫВОД

> Отсутствие логов приложения не означает отсутствие работы процесса.

Холодная страница машинного кода

ХОЛОДНЫЙ КОД

```
CPU выбирает инструкцию
↓ страница не резидентна
ошибка страницы
↓ страничный кеш?
├─ попадание: данные уже в RAM
└─ промах: чтение с накопителя
↓ продолжить инструкцию
```

ИНЖЕНЕРНЫЙ ВЫВОД

Исполняемый код также проходит через иерархию памяти и ввод-вывод.

Динамический исполняемый файл добавляет стартовый рабочий набор

ЗАДАЧА

Могут понадобиться страницы `ld.so`, libc, других библиотек, таблиц символов и релокаций.

МЕХАНИЗМ

Это новые чтения памяти и потенциальные ошибки страниц до пользовательского кода.

ИНЖЕНЕРНЫЙ ВЫВОД

Сравнивать холодный запуск нужно по реально затронутым страницам.

Релокации — конкретные обращения к памяти

СТОИМОСТЬ ЗАГРУЗЧИКА

```
прочитать таблицу релокаций
↓ найти символ или относительный адрес
↓ вычислить целевое значение
записать его в таблицу
↓ повторить для остальных записей
```

ИНЖЕНЕРНЫЙ ВЫВОД

На сложном динамическом приложении эта работа способна стать заметной частью запуска.

Ленивое связывание не устраняет стоимость

УПРОЩЕНИЕ

Оно может уменьшить стартовую работу, отложив разрешение части функций.

ТОЧНАЯ МОДЕЛЬ

Но первый вызов соответствующей функции затем оплачивает поиск и запись адреса.

ИНЖЕНЕРНЫЙ ВЫВОД

Системная задержка переносится между фазами жизненного цикла.

Контейнер добавляет уровни выше `execve`

КОНТЕЙНЕР

```
образ и слои файловой системы
↓ пространства имен / cgroup / запуск процесса
`execve`
↓ загрузчик ELF
↓ рантайм Go
↓ приложение и готовность
```

ИНЖЕНЕРНЫЙ ВЫВОД

4.5 анализирует именно участок `execve → точка входа`, не весь путь запуска pod.

Статический и distroless образ тоже имеют компромиссы

УПРОЩЕНИЕ

Плюсы: меньше динамических библиотек и проще набор исполняемых зависимостей.

ТОЧНАЯ МОДЕЛЬ

Минусы могут относиться к обновлению встроенного кода библиотек, отладке, системным данным и совместимости с cgo.

ИНЖЕНЕРНЫЙ ВЫВОД

Минимальность образа не гарантирует победу по всем метрикам.

Другая структура файла, та же задача

Mach-O описывает сегменты и зависимости, dyld подготавливает библиотеки и адреса.

```
Mach-O / LC_SEGMENT_64
сдвиг ASLR
dyld / библиотеки / общий кеш
```

На macOS нет ELF, но есть те же физические вопросы

УПРОЩЕНИЕ

Linux использует ELF, программные заголовки и `PT_LOAD`.

ТОЧНАЯ МОДЕЛЬ

macOS использует Mach-O и команды загрузки, но тоже должна создать отображения памяти, выбрать адреса, загрузить библиотеки и передать управление.

ИНЖЕНЕРНЫЙ ВЫВОД

Сопоставляем роли механизмов, а не названия полей.

Mach-O: заголовок и команды загрузки

MACH-O

```
заголовок Mach-O
↓ список команды загрузки
сегменты и их права
↓ требуемые dylib
↓ запись о точке входа
↓ сведения для подписания кода
```

ИНЖЕНЕРНЫЙ ВЫВОД

Команды загрузки дают загрузчику описание исполняемого образа.

LC_SEGMENT_64 — аналог роли PT_LOAD

СЕГМЕНТ

```
`LC_SEGMENT_64`
├─ `vmaddr` / `vmsize`
├─ `fileoff` / `filesize`
└─ `maxprot` / `initprot`
↓ создать виртуальное отображение
```

ИНЖЕНЕРНЫЙ ВЫВОД

Сегмент Mach-O связывает часть файла с виртуальным диапазоном и правами доступа.

__TEXT: код и данные только для чтения

ЗАДАЧА

Сегмент часто содержит исполняемый код и неизменяемые данные.

МЕХАНИЗМ

При загрузке назначаются соответствующие права; детали современной раскладки могут включать дополнительные сегменты.

ИНЖЕНЕРНЫЙ ВЫВОД

Чистые страницы кода могут отображаться как файловые страницы.

Данные Mach-O могут располагаться в нескольких сегментах

ЗАДАЧА

Записываемые глобальные значения требуют прав изменения, а часть метаданных можно защитить строже.

МЕХАНИЗМ

Поэтому современная раскладка делит данные на разные области.

ИНЖЕНЕРНЫЙ ВЫВОД

Разделение сегментов связано с защитой памяти и возможностями совместного использования.

сдвиг ASLR на macOS

СДВИГ АДРЕСОВ

```
в Mach-O известен предпочтительный адрес A
↓ ОС выбирает сдвиг S
фактический адрес = A + S
↓ dyld учитывает сдвиг при связывании
```

ИНЖЕНЕРНЫЙ ВЫВОД

Идея аналогична базе загрузки PIE, хотя формат и инструменты отличаются.

`otool -l` показывает команды загрузки

GO / КОМАНДА

```
otool -l ./app
```

ЧТО ПРОИСХОДИТ

Ищем `LC_SEGMENT_64`, `vmaddr`, `vmsize`, `fileoff`, права и записи о точке входа.

ИНЖЕНЕРНЫЙ ВЫВОД

Для macOS это основной аналог исследования сегментов ELF.

`otool -L` показывает библиотеки

GO / КОМАНДА

```
otool -L ./app
```

ЧТО ПРОИСХОДИТ

Выводит зависимости от `.dylib` — то, что при запуске обрабатывается dyld.

ИНЖЕНЕРНЫЙ ВЫВОД

Свойства бинарника macOS тоже нужно проверять по артефакту сборки.

Что делает dyld

ДИНАМИЧЕСКИЙ ЗАГРУЗЧИК

```
загрузчик получает Mach-O
↓ найти необходимые dylib
↓ использовать shared cache, где возможно
↓ привязки и исправления адресов
передать управление программе
```

ИНЖЕНЕРНЫЙ ВЫВОД

dyld решает сходные с ld-linux задачи, но на другом формате и со своими правилами.

Dyld shared cache оптимизирует системные библиотеки

ЗАДАЧА

Часть системного кода Apple находится в подготовленном общем кеше библиотек.

МЕХАНИЗМ

Это сокращает отдельные операции поиска и размещения и помогает делить системные страницы.

ИНЖЕНЕРНЫЙ ВЫВОД

Кеш dyld — механизм ОС, не часть исходного Go-приложения.

Подпись кода может проверяться до исполнения

ЗАДАЧА

Mach-O способен содержать `LC_CODE_SIGNATURE`; платформа проверяет допустимость запуска по своей политике безопасности.

МЕХАНИЗМ

Нельзя считать, что любые отображенные исполняемые байты сразу можно запускать.

ИНЖЕНЕРНЫЙ ВЫВОД

Безопасность — отдельная стадия жизненного цикла исполняемого файла.

Универсальный бинарник — один артефакт, несколько архитектур

УПРОЩЕНИЕ

На процессоре Apple Silicon может использоваться файл, содержащий варианты для `arm64` и `x86_64`.

ТОЧНАЯ МОДЕЛЬ

ОС выбирает нужную архитектурную часть. Затем уже загружает ее сегменты.

ИНЖЕНЕРНЫЙ ВЫВОД

Внешний формат может содержать несколько разных Mach-O-образов.

Общая модель ELF и Mach-O

ОБЩАЯ МОДЕЛЬ

```
исполняемый файл
↓ заголовки и описания сегментов
↓ виртуальные отображения / права
↓ ASLR
↓ библиотеки и исправления адресов
точка входа программы
```

ИНЖЕНЕРНЫЙ ВЫВОД

Имена форматов отличаются; причинная структура загрузки сохраняется.

Windows использует PE/COFF

ЗАДАЧА

PE содержит заголовки, секции, импортируемые функции, таблицы релокаций и точку входа.

МЕХАНИЗМ

Загрузчик Windows решает ту же задачу создания образа, хотя способ описания отличается от ELF и Mach-O.

ИНЖЕНЕРНЫЙ ВЫВОД

В лекции достаточно сравнения ролей, без отдельного разбора PE.

Основная цепочка Загрузчик Windows

WINDOWS

```
PE executable
↓ отображение образа
↓ загрузка DLL
↓ релокации базовых адресов
↓ разрешение импортов
переход в стартовый код
```

ИНЖЕНЕРНЫЙ ВЫВОД

Импорт и размещение библиотек — общая проблема разных ОС.

таблица адресов импорта (Import Address Table)

IAT

```
код использует функцию F из DLL
↓ загрузчик находит DLL и F
↓ записывает адрес в таблицу импортов
вызов идет по разрешенному адресу
```

ИНЖЕНЕРНЫЙ ВЫВОД

IAT решает задачу, сходную с таблицами адресов динамического ELF.

Проверяем файл, отображения и реальный запуск

Каждый инструмент отвечает на свой уровень: формат, библиотеки, виртуальная память и системные вызовы.

`readelf` → файл
`/proc/PID/maps` → процесс
`strace` / `LD_DEBUG` → события

`file`: первый взгляд на двоичный файл

GO / КОМАНДА

```
file ./app
```

ЧТО ПРОИСХОДИТ

Показывает формат, архитектуру, признаки динамической линковки, путь интерпретатора и наличие отладочной информации.

ИНЖЕНЕРНЫЙ ВЫВОД

За несколько секунд можно определить направление дальнейшего исследования.

`readelf -h`: тип и точка входа

GO / КОМАНДА

```
readelf -h ./app
```

ЧТО ПРОИСХОДИТ

Читаем `ET_EXEC`/`ET_DYN`, архитектуру, адрес входа и положение таблиц ELF.

ИНЖЕНЕРНЫЙ ВЫВОД

Заголовок отвечает на вопрос, что именно передано ядру.

`readelf -l`: сегменты и интерпретатор

GO / КОМАНДА

```
readelf -l ./app
```

ЧТО ПРОИСХОДИТ

Ищем `LOAD`, `INTERP`, `DYNAMIC`, `GNU_RELRO`, `GNU_STACK` и сочетания прав.

ИНЖЕНЕРНЫЙ ВЫВОД

Это главный источник для сопоставления файла с отображениями процесса.

`readelf -d`: динамические зависимости

GO / КОМАНДА

```
readelf -d ./app
```

ЧТО ПРОИСХОДИТ

Ищем `DT_NEEDED`, `DT_RUNPATH`, `DT_RPATH` и динамические флаги.

ИНЖЕНЕРНЫЙ ВЫВОД

Если запускается не та библиотека, начинаем с того, что записано в ELF.

`/proc/PID/maps`: реальная карта памяти

GO / КОМАНДА

```
cat /proc/$PID/maps
```

ЧТО ПРОИСХОДИТ

Видны диапазон адресов, права, файловое смещение и путь: исполняемый файл, `.so`, стек, vDSO и другие отображения.

ИНЖЕНЕРНЫЙ ВЫВОД

Показывает фактическое размещение после работы ядра и загрузчика.

Главное доказательство 4.5: ELF VM

ДОКАЗАТЕЛЬСТВО

```
`readelf -l`: `PT_LOAD`
↓ offset / виртуальный адрес / R-W-X

`/proc/$PID/maps`
↓ реальные диапазоны и смещения

сопоставить до и после запуска
```

ИНЖЕНЕРНЫЙ ВЫВОД

Формат файла напрямую превращается в наблюдаемые отображения.

`smaps` дополняет карту физической резидентностью

ЗАДАЧА

В `/proc/$PID/smaps` есть RSS, PSS, признаки частных и общих, а также файловых и анонимных страниц.

МЕХАНИЗМ

Но это уже слой виртуальной памяти из предыдущих лекций, а не отдельная механика ELF.

ИНЖЕНЕРНЫЙ ВЫВОД

Не смешиваем размер отображения и число физических страниц.

strace: что видно на границе системных вызовов

GO / КОМАНДА

```
strace -f -e
trace=execve,openat,mmap,mprotect,
munmap
./app
```

ЧТО ПРОИСХОДИТ

Можно увидеть запуск, открытие файлов библиотек, отображения загрузчика и изменение прав памяти.

ИНЖЕНЕРНЫЙ ВЫВОД

Это трассировка системных вызовов, а не внутреннего алгоритма ядра.

Почему `strace` не показывает все шаги ELF loader

УПРОЩЕНИЕ

Ожидание: на каждое отображение, которое ядро создало при `execve`, появится отдельный пользовательский `mmap`.

ТОЧНАЯ МОДЕЛЬ

Часть работы выполняется внутри реализации самого `execve` и не проявляется как отдельный вызов из пользовательского кода.

ИНЖЕНЕРНЫЙ ВЫВОД

Нужны и статический ELF-анализ, и карта памяти процесса.

`perf stat` для холодного запуска

GO / КОМАНДА

```
perf stat ./app
```

ЧТО ПРОИСХОДИТ

Сравниваем ошибки страниц, такты CPU, инструкции и время. Для очень короткого процесса требуется много повторов и контроль шумов.

ИНЖЕНЕРНЫЙ ВЫВОД

Холодный запуск измеряют как отдельную операцию, не по профилю CPU работающего сервиса.

Холодный и теплый запуски: что сравнивать

МЕТОДИКА

```
запуск 1 / запуск 2
↓ одинаковый исполняемый файл
длительность запуска
major / minor faults
↓ состояние страничного кеша
объяснить расхождение
```

ИНЖЕНЕРНЫЙ ВЫВОД

Очистка системного кеша допустима лишь в лабораторном окружении, не на рабочем сервере.

LD_DEBUG показывает работу glibc loader

GO / КОМАНДА

```
LD_DEBUG=libs,reloc ./app
```

ЧТО ПРОИСХОДИТ

Для динамического ELF можно увидеть выбор библиотек и применение релокаций. Для статической программы соответствующего загрузчика может не быть.

ИНЖЕНЕРНЫЙ ВЫВОД

Диагностика зависит от фактического режима линковки.

macOS: сегменты и память после загрузки

GO / КОМАНДА

```
otool -l ./app
otool -L ./app
vmmap $PID
```

ЧТО ПРОИСХОДИТ

Первый инструмент показывает команды загрузки, второй — зависимости от dylib, третий — отображения уже работающего процесса.

ИНЖЕНЕРНЫЙ ВЫВОД

Те же три вопроса: файл библиотеки память.

Диагностика dyld имеют платформенные ограничения

ЗАДАЧА

Переменные диагностики dyld, например `DYLD_PRINT_LIBRARIES`, помогают увидеть загруженные библиотеки.

МЕХАНИЗМ

Поведение зависит от версии macOS и политики безопасности процесса.

ИНЖЕНЕРНЫЙ ВЫВОД

Нельзя считать диагностические переменные универсально работающими для любой программы.

Собрать загрузить увидеть объяснить

Эксперименты проверяют ELF-сегменты, страницы, адреса, динамические библиотеки и точку входа Go.

```
readelf / maps / smaps
PIE / Go без cgo / cgo
strace / dyld / холодный запуск
```

Практика 1. ELF-заголовок

GO / КОМАНДА

```
CGO_ENABLED=0 go build -o app .
readelf -h ./app
```

ЧТО ПРОИСХОДИТ

Найти архитектуру, тип ELF, адрес входа и таблицу программных заголовков.

ИНЖЕНЕРНЫЙ ВЫВОД

Файл содержит все базовые сведения для его распознавания ядром.

Практика 2. Секции против сегментов

GO / КОМАНДА

```
readelf -S ./app
readelf -l ./app
```

ЧТО ПРОИСХОДИТ

Связать `.text`, `.rodata`, `.data`, `.bss` с `PT_LOAD`, учитывая группировку секций по правам.

ИНЖЕНЕРНЫЙ ВЫВОД

Взгляд компоновщика и загрузчика устроен по-разному.

Практика 3. Большая `.bss` и маленький файл

GO / КОМАНДА

```
var big [512 << 20]byte

readelf -l ./app
ls -lh ./app
```

ЧТО ПРОИСХОДИТ

Сравнить размер файла и размер отображаемого диапазона. Добавить использование массива, чтобы компилятор не удалил ненужный символ.

ИНЖЕНЕРНЫЙ ВЫВОД

Нулевые байты в памяти не обязаны храниться в файле.

Практика 4. PT_LOAD и карта процесса

GO / КОМАНДА

```
readelf -l ./app
cat /proc/$PID/maps
```

ЧТО ПРОИСХОДИТ

Удерживать процесс живым и сопоставить смещения, права, виртуальные адреса и имя файла.

ИНЖЕНЕРНЫЙ ВЫВОД

Связь ELF VM доказана на реально работающем процессе.

Практика 5. Отображение еще не резидентность

НАБЛЮДЕНИЕ

```
большой глобальный массив `.bss`
↓ сначала не читать / не менять
замерить RSS и ошибки страниц
↓ обратиться к страницам по одной
замерить повторно
```

ИНЖЕНЕРНЫЙ ВЫВОД

Физическая память расходуется в зависимости от обращения к страницам.

Практика 6. PIE и ASLR

GO / КОМАНДА

```
go build -o app-normal .
go build -buildmode=pie -o
app-pie .
```

ЧТО ПРОИСХОДИТ

Запускать повторно и сравнивать базы образа через `/proc/PID/maps`. Проверить тип ELF.

ИНЖЕНЕРНЫЙ ВЫВОД

Позиционно-независимый образ допускает изменение базового адреса.

Практика 7. Чистый Go против cgo

GO / КОМАНДА

```
CGO_ENABLED=0 go build -o pure .
file ./pure
readelf -l ./pure
ldd ./pure
```

ЧТО ПРОИСХОДИТ

Сравнить с программой, которая действительно использует cgo. Ищем `INTERP`, `NEEDED` и внешние библиотеки.

ИНЖЕНЕРНЫЙ ВЫВОД

Свойства загрузки зависят от конфигурации сборки.

Практика 8. Отдельная работа динамического загрузчика

GO / КОМАНДА

```
LD_DEBUG=libs,reloc ./dynamic
```

ЧТО ПРОИСХОДИТ

Для динамической сборки найти поиск библиотек и релокации в трассировке `ld-linux`.

ИНЖЕНЕРНЫЙ ВЫВОД

Загрузчик работает до передачи управления стартовому коду программы.

Практика 9. Запуск через `strace`

GO / КОМАНДА

```
strace -f -e
trace=execve,openat,mmap,mprotect
./dynamic
```

ЧТО ПРОИСХОДИТ

Сравнить статическую и динамическую сборку по обращениям к файловой системе, отображениям и правам.

ИНЖЕНЕРНЫЙ ВЫВОД

Не путать системные вызовы `ld-linux` с внутренними действиями ядра при `execve`.

Практика 10. Источник `os.Args`

GO / КОМАНДА

```
func main() {
    fmt.Printf("%q\n", os.Args)
}

./app a b c
```

ЧТО ПРОИСХОДИТ

Проследить аргументы от оболочки через `execve`, стартовый стек и рантайм до пакета `os`.

ИНЖЕНЕРНЫЙ ВЫВОД

`os.Args` появляется из данных, переданных ядром при запуске.

Практика 11. Точка входа

GO / КОМАНДА

```
readelf -h ./app
go tool objdump ./app
```

ЧТО ПРОИСХОДИТ

Взять `Entry point address` и сопоставить стартовый участок кода с `rt0_*`. Не начинать сразу с `main.main`.

ИНЖЕНЕРНЫЙ ВЫВОД

Между ОС и пользовательским `main` есть обязательная загрузка рантайма.

Практика 12. Холодный и прогретый запуск

ИЗМЕРЕНИЕ

```
короткоживущая программа
↓ серия запусков и замеров
время / major faults / minor faults
↓ сравнить влияние кеша
объяснить результат без опасных действий
```

ИНЖЕНЕРНЫЙ ВЫВОД

Задержка запуска зависит от памяти и файловой системы, даже если сам Go-код прост.

Практика 13. Разделяемые страницы `.so`

GO / КОМАНДА

```
cat /proc/$PID/maps
cat /proc/$PID/smaps
```

ЧТО ПРОИСХОДИТ

Запустить несколько экземпляров динамической программы и посмотреть общие чистые страницы и PSS.

ИНЖЕНЕРНЫЙ ВЫВОД

RSS одного процесса не равен общему физическому потреблению системы.

Практика 14. Mach-O и dyld

GO / КОМАНДА

```
otool -l ./app
otool -L ./app
vmmap $PID
```

ЧТО ПРОИСХОДИТ

Сопоставить LC_SEGMENT_64, зависимости библиотек и отображения на macOS.

ИНЖЕНЕРНЫЙ ВЫВОД

Роли загрузки одинаковы, хотя интерфейсы инструментов и формат отличаются.

Когда ELF и загрузчик ломают backend-сервис

Практическая диагностика ошибок интерпретатора, отсутствующих библиотек и медленного старта.

```
есть файл, но ENOENT
не найдена `.so`
рост p99 запуска
```

Файл существует, но `execve` сообщает `No such file or directory`

УПРОЩЕНИЕ

Интуитивное объяснение: путь к самому исполняемому файлу неправильный.

ТОЧНАЯ МОДЕЛЬ

Если ELF требует `PT_INTERP`, но указанный динамический интерпретатор отсутствует в видимой файловой системе, запуск также может завершиться `ENOENT`.

ИНЖЕНЕРНЫЙ ВЫВОД

Проверяем интерпретатор, а не только существование основного файла.

Классический сбой в минимальном контейнере

КОНТЕЙНЕР

```
собрали ELF против glibc
↓ `PT_INTERP` указывает `/lib64/ld-linux-*`
перенесли файл в минимальный контейнер
↓ загрузчика по пути нет
`execve` не может начать программу
```

ИНЖЕНЕРНЫЙ ВЫВОД

Не всякая ошибка запуска принадлежит Go или Docker; она может возникать в загрузчик ELF в ядре.

Как найти отсутствующий интерпретатор

GO / КОМАНДА

```
file ./app
readelf -l ./app
```

ЧТО ПРОИСХОДИТ

В записи `INTERP` проверить путь к загрузчику, затем проверить существование этого пути внутри файлового пространства контейнера.

ИНЖЕНЕРНЫЙ ВЫВОД

Диагностика должна проходить в той же файловой среде, где запускается сервис.

Библиотека не найдена: другая фаза ошибки

ОШИБКА ЗАВИСИМОСТИ

```
`PT_INTERP` найден
↓ `ld-linux` начал работу
читает `DT_NEEDED`
↓ ищет нужную `.so`
библиотеки нет в путях поиска
↓ ошибка динамического загрузчика
```

ИНЖЕНЕРНЫЙ ВЫВОД

Отсутствие интерпретатора и отсутствие библиотеки — разные этапы запуска.

Разбираем `shared libraries` ошибку

GO / КОМАНДА

```
readelf -d ./app
ldd ./app
LD_DEBUG=libs ./app
```

ЧТО ПРОИСХОДИТ

Ищем нужную библиотеку и каталог, в котором ее пытается найти загрузчик. Для неизвестного файла предпочитаем `readelf`.

ИНЖЕНЕРНЫЙ ВЫВОД

Сначала подтверждаем динамическую зависимость, затем исследуем путь поиска.

Сервис долго стартует до первых логов

ПОИСК ФАЗЫ

```
до `main.main`
├─ файловая система / загрузчик
├─ библиотеки и релокации
├─ ошибки страниц
├─ первичная инициализация Go
└─ инициализация пакетов
```

ИНЖЕНЕРНЫЙ ВЫВОД

Для локализации нужно разделить работу загрузчика и рантайма, а не смотреть только HTTP pprof.

Время динамического загрузчика и `init()` Go нельзя смешивать

УПРОЩЕНИЕ

Симптом один: долгий интервал до пользовательского `main`.

ТОЧНАЯ МОДЕЛЬ

Причины могут находиться в разных программах и слоях: `ld-linux` до точки входа или рантайм Go после нее.

ИНЖЕНЕРНЫЙ ВЫВОД

Результаты `strace` и `inittrace` отвечают на разные вопросы.

Запуск pod и задержка HTTP-запроса — разные SLO

ЗАДАЧА

Холодный запуск может ухудшиться после поставки новой сборки, когда работающий сервис отвечает с прежней задержкой.

МЕХАНИЗМ

Прогретый профиль CPU не обязательно покажет чтение библиотек и ошибки страниц при старте.

ИНЖЕНЕРНЫЙ ВЫВОД

Измеряем p99 времени запуска отдельно от p99 задержки запросов.

Нельзя выбирать static или dynamic по лозунгу

УПРОЩЕНИЕ

«Статика всегда быстрее» и «динамика всегда экономит RAM» — слишком общие заявления.

ТОЧНАЯ МОДЕЛЬ

Сравниваем совместимость, обновление библиотек, размер образа, запуск, PSS/RSS, наличие cgo и возможности диагностики.

ИНЖЕНЕРНЫЙ ВЫВОД

Выбор линковки должен соответствовать эксплуатационным ограничениям.

Статическая сборка меняет процесс обновления библиотек

ЗАДАЧА

Код зависимости встроен в бинарник, поэтому для исправления библиотеки может понадобиться новая сборка и поставка приложения.

МЕХАНИЗМ

При динамической линковке совместимую `.so` можно обновить отдельно, но возникает требование поддерживать ABI и целевую среду.

ИНЖЕНЕРНЫЙ ВЫВОД

Цена независимости бинарника — более явная ответственность за обновления.

Почему статический Go-артефакт удобнее переносить

ЗАДАЧА

Меньше зависимостей от внешних путей библиотек и конкретного динамического загрузчика.

МЕХАНИЗМ

Но это не отменяет вопросов сертификатов, локали, системных данных, отладки и поведения отдельных пакетов.

ИНЖЕНЕРНЫЙ ВЫВОД

Самодостаточный ELF не означает отсутствия любых внешних ресурсов.

ASLR не исправляет повреждение памяти

УПРОЩЕНИЕ

ASLR меняет адреса кода и данных, делая часть атак менее надежной.

ТОЧНАЯ МОДЕЛЬ

Уязвимое обращение к памяти остается уязвимым. Защита снижает последствия, но не заменяет корректность.

ИНЖЕНЕРНЫЙ ВЫВОД

Рандомизация адресов — смягчающая мера.

NX, W^X и RELRO защищают разные вещи

ЗАЩИТА ПАМЯТИ

```
NX → нельзя исполнять страницы данных
↓ W^X → не совмещать запись и исполнение
↓ RELRO → защитить таблицы после релокаций
```

ИНЖЕНЕРНЫЙ ВЫВОД

Эти механизмы дополняют друг друга, но не являются взаимозаменяемыми.

Динамический загрузчик не участвует в каждом HTTP-запросе

УПРОЩЕНИЕ

Наивная модель: если используется `.so`, ее символы постоянно ищутся по строковым именам.

ТОЧНАЯ МОДЕЛЬ

После релокаций адреса уже готовы. Исключение — первый вызов функции при разрешенном ленивом связывании.

ИНЖЕНЕРНЫЙ ВЫВОД

Разовая стоимость запуска и цена горячего пути различаются.

Что гарантирует формат, а что зависит от ОС

Отделяем устойчивый путь загрузки от подробностей реализации ядра и динамического загрузчика.

```
ELF ABI / execve
политика ASLR / реализация загрузчика
инструменты и метрики
```

Устойчивый механизм загрузки

ЗАДАЧА

Исполняемый файл задает формат и сегменты. Загрузчик создает отображения, назначает права и передает управление стартовому коду.

МЕХАНИЗМ

При наличии общих библиотек может выполняться дополнительная работа по поиску и связыванию.

ИНЖЕНЕРНЫЙ ВЫВОД

Эта причинная модель переносима между разными платформами.

Что задает Linux/ELF ABI

ЗАДАЧА

Например `e_entry`, таблица программных заголовков, `PT_LOAD`, `PT_INTERP`, `PT_DYNAMIC`, `DT_NEEDED`, `execve`, `auxv`.

МЕХАНИЗМ

Эти структуры и интерфейсы позволяют связать готовый файл с процессом и его пользовательским начальным состоянием.

ИНЖЕНЕРНЫЙ ВЫВОД

Формат загрузки — контракт платформы, не деталь исходного Go-кода.

Что остается деталью реализации

ЗАДАЧА

Политика ASLR, внутренние шаги ядра, алгоритм поиска библиотек, оптимизации `ld-linux`, детали dyld shared cache.

МЕХАНИЗМ

Они могут меняться между версиями ОС, архитектурами и вариантами загрузчика.

ИНЖЕНЕРНЫЙ ВЫВОД

В диагностике фиксируем целевую платформу и версии.

Четыре различия, которые нельзя терять

УПРОЩЕНИЕ

Секции ≠ сегменты; отображено ≠ физически резидентно.

ТОЧНАЯ МОДЕЛЬ

Исполняемый файл ≠ работающий процесс; точка входа ≠ `main.main`.

ИНЖЕНЕРНЫЙ ВЫВОД

Эти различия предотвращают самые распространенные ошибки модели.

Полная цепочка статического Linux ELF

СТАТИЧЕСКИЙ ПУТЬ

```
оболочка → создание процесса
↓ `execve` / ядро распознает ELF
↓ `PT_LOAD` и права R/W/X
↓ начальный стек / `auxv` / vDSO
↓ `e_entry`
стартовый код Go
```

ИНЖЕНЕРНЫЙ ВЫВОД

Нет отдельного пользовательского этапа загрузки общих библиотек.

Полная цепочка динамического Linux ELF

ДИНАМИЧЕСКИЙ ПУТЬ

`execve` → ELF / `PT_INTERP`
↓ загрузить `ld-linux`
↓ обработать `DT_NEEDED`
↓ отобразить `.so`
↓ символы / релокации / RELRO
↓ точка входа Go

ИНЖЕНЕРНЫЙ ВЫВОД

Динамический загрузчик работает до передачи управления стартовому коду программы.

Полная цепочка холодной страницы кода

PAGE FAULT

```
страница есть в виртуальном адресном плане
↓ CPU обращается к ней впервые
ошибка страницы
↓ страничный кеш или накопитель
↓ обновить таблицу страниц
повторить команду CPU
```

ИНЖЕНЕРНЫЙ ВЫВОД

Виртуальное отображение и физическая резидентность возникают в разные моменты.

Полная цепочка ASLR + PIE

ПЕРЕМЕЩЕНИЕ

```
позиционно-независимый ELF
↓ ядро назначает случайную базу B
адрес F = B + смещение F
↓ разрешить оставшиеся ссылки
↓ выполнить стартовый код
```

ИНЖЕНЕРНЫЙ ВЫВОД

PIE делает основную программу перемещаемой, ASLR выбирает ее положение.

Полная цепочка первого вызова через PLT

ЛЕНИВОЕ СВЯЗЫВАНИЕ

```
`CALL F@PLT`
↓ GOT пока ведет к resolver
↓ найти адрес F в библиотеке
↓ записать адрес в GOT
повторный вызов → готовый адрес
```

ИНЖЕНЕРНЫЙ ВЫВОД

Ленивое связывание платится при первом вызове, не при каждом обращении.

Полная цепочка Mach-O

MACOS

```
Mach-O и команды загрузки
↓ `LC_SEGMENT_64` → виртуальная память
↓ сдвиг ASLR
↓ dyld / dylib / исправления адресов
точка входа
↓ стартовый код Go
```

ИНЖЕНЕРНЫЙ ВЫВОД

На macOS та же задача решается своими структурами файла и загрузчиком.

Финальная инженерная мысль

ЗАДАЧА

Компоновщик заканчивает построение файла. Ядро превращает его в адресное пространство. Динамический загрузчик при необходимости связывает библиотеки.

МЕХАНИЗМ

Только после этого процесс может приступить к стартовому коду рантайма Go.

ИНЖЕНЕРНЫЙ ВЫВОД

Оценивать запуск приложения нужно по всей вертикали, а не только от первой строки `main`.

Файл процесс стартовый код Go

Теперь понятно, какие механизмы работают до появления пользовательского `main.main`.

```
ELF / Mach-O → отображения
`execve` / загрузчик → адреса
следующий слой: `rt0` и первичная инициализация Go
```